

Part 2: Agent Foundations & Concepts

Understanding the Foundation of Agentic Coding Tools

Agentic Coding Tools Series

University of South Carolina

January 23, 2026

Overview

- 1 What IS an Agent?
- 2 Core Agent Components
- 3 The ReAct Loop
- 4 Agent Specialization
- 5 Context Windows Deep Dive
- 6 Agent Autonomy Levels
- 7 Summary & Key Takeaways

Formal Definition: What is an Agent?

An agent is an autonomous software system that:

1. **Perceives** its environment through sensors/tools
2. **Reasons** about the current state and goals
3. **Takes Actions** to influence the environment
4. **Observes** the results of those actions
5. **Adapts** behavior based on feedback

Mathematical Definition

$$\text{Agent} = \langle P, S, A, \delta, \gamma, G \rangle$$

P Perception (observations)

S State space

A Action space

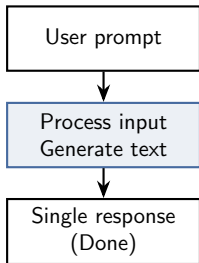
δ State transition function

γ Goal evaluation function

G Goal specification

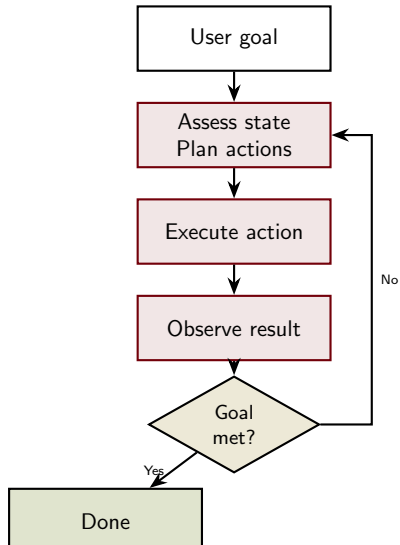
Agent vs Traditional AI Assistant

TRADITIONAL AI ASSISTANT



VS

AGENT



Key Characteristics of Agents

#1 Autonomy

The ability to make decisions and take actions without constant user direction

Spectrum:

- Manual (Chatbot)
- Semi-Autonomous
- Fully Autonomous

#2 Goal-Directed

Behavior determined by explicit objectives, not instructions

Example:

- “Minimize test runtime to under 100ms”
- Agent finds its own approach

#3 Tool Use

Agents act on environments through well-defined tools

Without tools:

Agent can only think

With tools:

Agent can think AND do

Why Agents Matter for Software Development

Traditional Development:

- Write code
- Run tests
- Fix errors
- Repeat

Human-intensive, one decision at a time

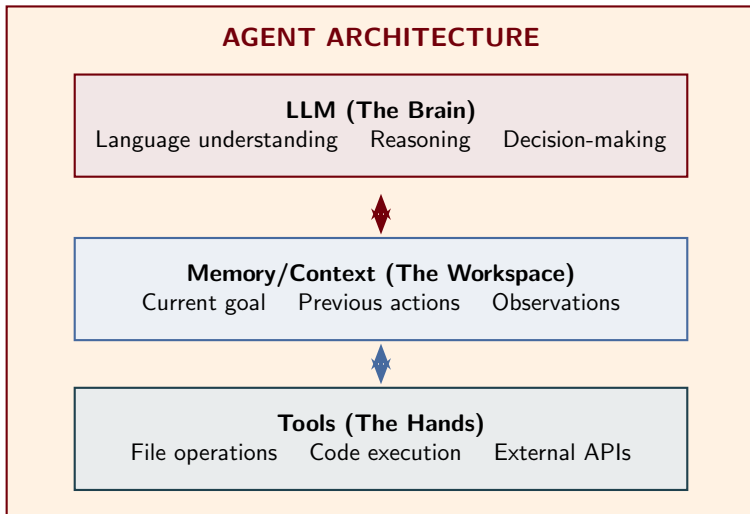
Agentic Development:

- Agent: Implement feature
 - Write code
 - Run tests
 - Fix failures
 - Optimize performance
 - Add documentation
 - Submit for review

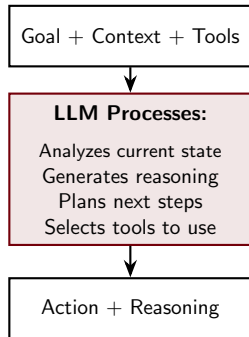
Benefits

- Parallelizable tasks (agent explores while humans plan)
- Tedious tasks automated (debugging, testing)
- 24/7 availability and consistent process execution

Core Agent Components: The Architecture



Component #1: The LLM — The Brain



LLM Characteristics That Matter:

- Quality of reasoning
- Instruction-following ability
- Tool use capability
- Consistency (reliability)
- Speed vs capability tradeoff

Model Selection

Routine coding	Claude Haiku
Complex tasks	Claude Opus
Cost-sensitive	Llama3

Component #2: Memory & Context — The Workspace

What Lives in Memory:

- **Goal:** “Implement pagination for user API”
- **Observations:** Current code structure, previous test results, errors encountered
- **Reasoning History:** Steps tried, why they failed/succeeded
- **Plan:** Next actions, fallback strategies

Key Insight:

Memory = Context window usage

More memory = Better decisions

But: Context is *finite and expensive*

Context Window Sizes (2026)

Claude Opus 4.5	200K tokens
GPT-4 Turbo	128K tokens
Claude Haiku	200K tokens
Gemini Pro 1.5	1M tokens

Management Strategies:

1. Summarization
2. Archival to files
3. Forgetting old context
4. Compression
5. Phasing tasks

Tool Count Sweet Spot

Too few (3–5): Agent paralyzed

Optimal (8–15): Effective
Confused

Too many (50+):

Component #4: Planning Capability

Non-Planning Agent:

1. Read file
2. Write changes
3. Test
4. Fails!
5. Back to 1
6. Loop 5 times...

Cost: High, Time: Long

Planning Agent:

1. PLAN:

- Map structure
- Identify patterns
- Draft changes

2. Write smart

3. Test

4. Success!

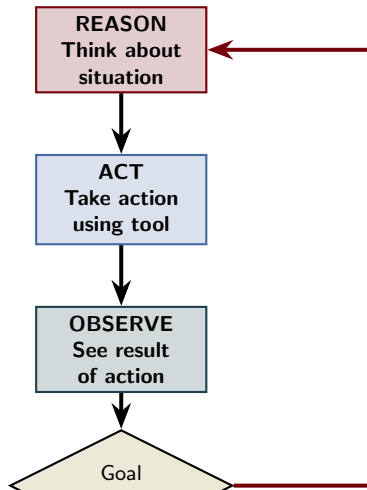
Cost: Lower, Time: Shorter

Planning Techniques

- **Hierarchical:** Break high-level goals into sub-tasks
- **Contingency:** IF condition THEN approach A ELSE approach B
- **Dependency:** Understand which changes affect which components

The ReAct Loop: Core Agent Operating Pattern

ReAct = REASONing + ACTing



The fundamental pattern
all modern agents use.

Reasoning and action are
interleaved, not sequential.

~~No: Loop back~~
Agent doesn't plan everything
upfront—it **adapts** as it goes.

REASON Step: Thinking About the Situation

Input to REASON:

- Current goal: “Fix failing tests in auth module”
- What we know so far:
 - Test failures from previous step
 - Code we’ve examined
 - Changes we’ve made
- What we still need:
 - More information?
 - To take action?
 - To change something?

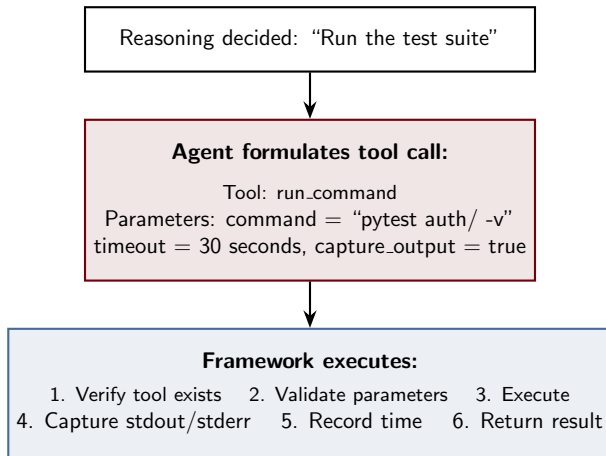
LLM’s Thinking Process

“I see test X failed with error Y. The error looks like Z (connection refused). That suggests the database isn’t running. I should check if database service is started. If not running, I’ll start it. If running, I’ll examine the connection string.”

Output of REASON:

- Decision: What to do next
- Reasoning: Why that’s right
- Expected outcome: What we’ll learn

ACT Step: Taking Action



Key Insight

Action is the only step that changes the world

OBSERVE Step: Perceiving Results

System Output

```
$ pytest auth/ -v  
  
test_login_valid_user ... PASS  
test_login_invalid_user ... PASS  
test_session_timeout ... FAIL  
    AssertionError: Session not expired  
  
2 passed, 1 failed in 2.34s
```

Agent's Observation

"2 passed, 1 failed. The failure is in test_session_timeout and the message is 'Session not expired'. This suggests the timeout logic isn't working."

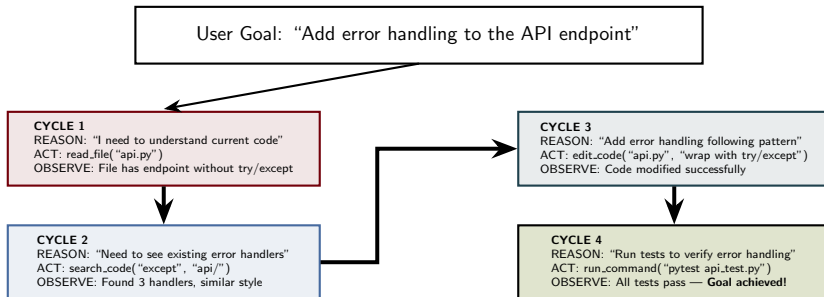
Good Observation includes:

- ✓ What succeeded (2 tests passed)
- ✓ What failed (1 test failed)
- ✓ Why it failed (timeout not working)
- ✓ What to do next

Poor Observation:

- ✗ Just reports: "Tests ran"
- ✗ Misses details
- ✗ Doesn't extract next step

Complete ReAct Cycle: An Example



Why ReAct Works: Comparison

Waterfall Planning

1. Understand all requirements
2. Create complete plan
3. Execute plan steps
4. If step N fails → back to 1

Can't predict all requirements

$O(n^2)$ failures

Reactive (No Loop)

1. Take next action
2. See result
3. Repeat

No forward-thinking

$O(n)$ wrong steps

ReAct (Interleaved)

1. Think about situation
2. Take one action
3. Observe result
4. Use result to inform next

Adaptive + thoughtful

$O(\log n)$ iteration

ReAct Advantage

By making one change, observing the result, and using that observation to inform the next decision, agents avoid the cascading failures of pure planning.

Generalists vs Specialists

Generalist Agent

“Do whatever task is needed”

Can handle: Coding, Testing, Deployment, Documentation

Problems:

- ✗ Slower (tries many approaches)
- ✗ Less accurate
- ✗ Context bloat
- ✗ Hard to debug

Specialist Agent

“Write Python tests for this code”

Specialized for: One language/framework, one type of task

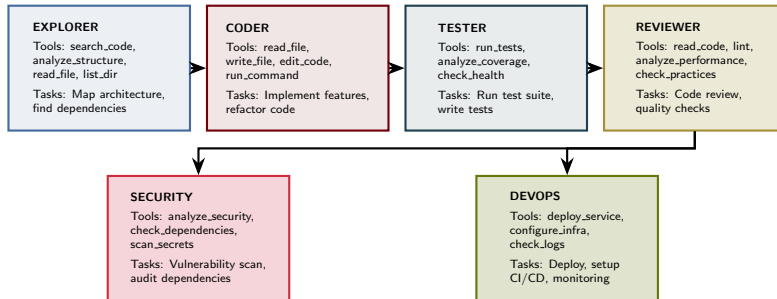
Advantages:

- ✓ Fast (fewer options)
- ✓ Accurate (deep expertise)
- ✓ Lean context
- ✓ Easy to debug

Modern Best Practice

Use **multiple specialists + coordinator** instead of one generalist

Common Specialist Agent Types



Design Principles for Custom Specialists

1. Define Clear Responsibility

- Specialist does ONE thing really well
- NOT: “Handle all database operations”
- YES: “Write SQL queries for this specific schema”

2. Specify Exact Tools

- Only the tools needed for this role
- NOT: “Use all tools” YES: “Use these 5 tools”

3. Create Specialized Prompt

- “You are an expert in PostgreSQL. Your role is to write optimal queries...”
- Include domain-specific best practices

4. Test Against Known Tasks

- Verify specialist works on typical tasks
- “Write query for active users”, “Optimize slow report query”

Key Principles

Narrow is better than broad
detailed

Specialization reduces error rate

Prompts should be

Context Windows: The Technical Foundation

Agent's Context Window

System prompt (1-2K)

User request (100-500)

Code files (3K+)

Previous history (1-5K)

Tool descriptions (1.5K)

Agent reasoning (1-5K)

What are Tokens?

- 1 token $\approx$ 0.75 words (English)
- 1 token $\approx$ 4 characters
- 1 page $\approx$ 500–800 tokens

Window Size by Model (2026)

Claude Haiku	200K
Claude Opus 4.5	200K
GPT-4 Turbo	128K
Gemini Pro 1.5	1M

200K tokens $\approx$ 150 pages

Token Limits: Theory vs Practice

Theoretical:

- Claude Opus: 200K tokens capacity
- = 150 pages of text
- “Should be enough for any task”

Practical Consumption:

- System prompt: 2K tokens
- Goal specification: 1K tokens
- Code files: 20K tokens
- Tool descriptions: 2K tokens
- Conversation history: 10K tokens

= 35K tokens before agent even starts working!

Agent reasoning consumes space:

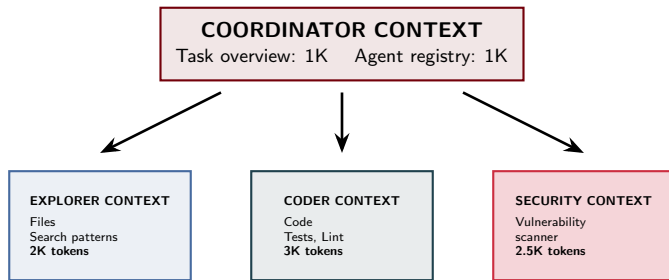
- Each decision: 500–2K tokens
- Each observation: 1K–5K tokens
- 10 cycles: ~30K tokens

Practical Rule of Thumb

Use no more than **50%** of context window for “setup” (prompts, files, history)

If using >80%, agent will run out of space mid-task and fail

Context Isolation: Multi-Agent Architecture



Benefits: ✓ Each agent has exactly needed context ✓ Reduced total token usage
✓ Can parallelize (agents don't interfere) ✓ Easy to reason about (focused scope)

Context Management Strategies

Strategy 1: Chunking

- Load only relevant file at a time
- Cycle 1: Load `user_routes.py`, refactor, test, save
- Cycle 2: Load `auth_routes.py`, refactor, test, save
- Cleaner, parallelizable

Strategy 2: Summarization

- 10K-token class → 1K summary
- Keep method signatures, drop implementation details
- Saves 9K tokens

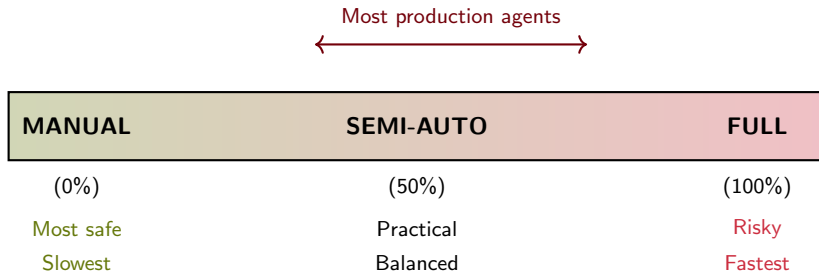
Strategy 3: Phasing

- Phase 1: “Fix critical bugs”
- Phase 2: “Add type hints”
- Phase 3: “Optimize performance”
- Each phase: clean context, focused goal

Strategy 4: Lazy Loading

- Don't load file until needed
- Agent requests → load
- Avoids wasting tokens on unneeded content

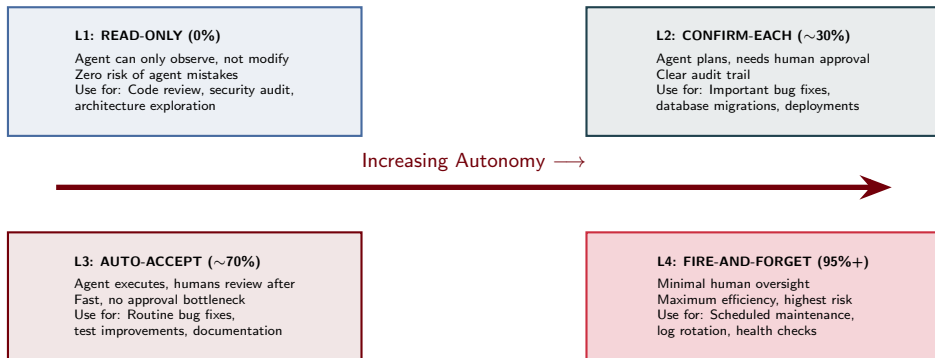
Autonomy Levels: Balancing Safety & Efficiency



Why This Matters:

- Higher autonomy = faster execution
- Lower autonomy = more control/safety
- Different tasks need different levels

The Four Autonomy Levels



Choosing the Right Autonomy Level

Key Questions:

1. What's the cost of failure?

Low cost → Higher autonomy

2. Can we detect failure?

Easy detection → Higher autonomy

3. Can we recover quickly?

Easy recovery → Higher autonomy

4. Do we trust the agent?

High trust → Higher autonomy

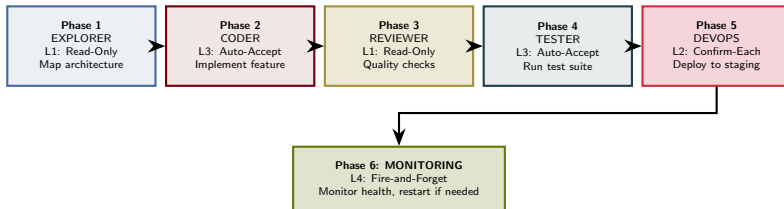
Selection Guide

Task Type	Level
Security fixes	L2
Performance bugs	L3
Documentation	L3
Test generation	L3
Code review	L1
Deployments	L2
Exploration	L1
Maintenance	L4

Best Practice

Start conservative, increase over time. Different agents at different levels. Match autonomy to task risk.

Real-World Project: Mixed Autonomy Levels



Pattern: Different agents at different levels in a coordinated pipeline

Part 2 Complete: Agent Foundations

What We Covered:

■ Section 1: What IS an Agent?

- Formal definition
- Agent vs traditional AI
- Key characteristics

■ Section 2: Core Components

- LLM (Brain)
- Memory/Context (Workspace)
- Tools (Hands)
- Planning

■ Section 3: The ReAct Loop

- REASON → ACT → OBSERVE
- Why it works

■ Section 4: Agent Specialization

- Specialists vs Generalists
- Common agent types
- Design principles

■ Section 5: Context Windows

- Token arithmetic
- Theory vs practice
- Management strategies

■ Section 6: Autonomy Levels

- L1 through L4
- Choosing the right level
- Real-world mixing

Foundational Principles

The Five Core Principles

1. Agents are iterative, not one-shot

The ReAct loop: REASON $\rightarrow$ ACT $\rightarrow$ OBSERVE $\rightarrow$ repeat

2. Specialization beats generalization

Multiple focused agents $>$ One generalist agent

3. Context is finite and expensive

50% for setup, 50% for work; manage strategically

4. Autonomy must match task risk

Start conservative, increase as trust builds

5. Tools determine capability

Well-designed tools = effective agents; 8–15 is optimal

$$\text{Agent Capability} = \text{LLM Quality} \times \text{Tool Design} \times \text{Memory Management} \times \text{Planning}$$

Self-Check: Can You Explain...

1. To a non-technical person:

What an agent is and why it's different from ChatGPT?

2. Walk through a ReAct loop for “fix this bug”:

What happens in REASON, ACT, OBSERVE?

3. Design a specialist agent for writing unit tests:

What tools would it have? What would it NOT have?

4. You have a 200K token context, project needs 30K tokens:

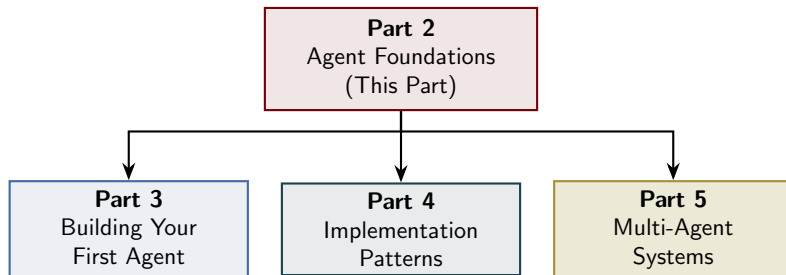
How would you manage this?

5. A task could be done at L2 or L3:

What questions would you ask to decide?

If you can answer these, you understand Part 2.

Coming Up Next...



From Theory to Practice:

- Part 3: Implement your first Explorer + Coder agents
- Part 4: Common patterns (pipelines, parallel execution, feedback loops)
- Part 5: Agent communication, shared state, emergent behavior

Questions?

Part 2: Agent Foundations & Concepts

Agentic Coding Tools Series

University of South Carolina